Faculdade de Engenharia da Universidade do Porto

IEC 61499 Compliant Web-based IDE for Function Block Design

Pedro da Cunha Teixeira e Faro Beirão

Working version

FEUP FACULDADE DE ENGENHARIA UNIVERSIDADE DO PORTO

Mestrado em Engenharia Informática e Computação

Supervisor: Prof. Rui Pinto

May 23, 2026

"Programming is not a zero-sum game. Teaching something to a fellow programmer doesn't take it away from you. I'm happy to share what I can, because I'm in it for the love of programming."

John Carmack

Contents

List of Figures

List of Tables

List of Listings

List of Acronyms

CPPS Cyber-Physical Production Systems

CRDT Conflict-free Replicated Data Type

FBD Function Block Diagram

ICS Industrial Control Systems

IDE Integrated Development Environment

PLC Programmable Logic Controller

RTE Runtime Environment

SOA Service-Oriented Architectures

UI User Interface

Chapter 1

Introduction

1.1 Context

Cyber-Physical Production Systems (CPPS) [7] are complex systems that integrate physical processes with digital technologies to optimise production processes. Several software engineering approaches and technologies are utilised in the development and operation of CPPS. One of such approaches is Function Block Diagram (FBD) [2], which are graphical programming languages used in industrial automation and control systems. FBD uses graphical function blocks to represent the logic and control of a system. The function blocks are connected in a network to form a control program. One of the benefits of FBD is that it provides a visual representation of the control logic, which can be easier to understand and debug than traditional textual programming languages. FBD is also modular, which means that function blocks can be reused in multiple programs, reducing development time and improving code maintainability. Also, FBD is often used in conjunction with Programmable Logic Controllers (PLCs) to control machinery and processes in manufacturing and other industrial applications [8, 9]. FBD programs are not as simple to create and edit as traditional text-based languages since they require the use of specialised software tools that allow for the manipulation of function blocks [17].

In industrial automation, it is common to use the *IEC 61499* standard, which defines an event-driven FBD approach for designing control software [12]. Unlike traditional PLC programming languages, *IEC 61499* emphasises modularity, reusability, and explicit separation of events and data, allowing developers to design complex distributed systems in a scalable and maintainable way. Function blocks define both algorithms and interfaces for inputs, outputs, and events, making them ideal for CPPS development [3]. This approach is particularly suitable for distributed automation systems, where control logic must respond to events generated by multiple devices or sensors in real time. An example of such a function block interface is shown in Figure 1.1, highlighting the event inputs, outputs, and data connections that enable flexible interaction between system components.

While Runtime Environments (RTEs) enable the execution of function blocks compliant with the standard, Integrated Development Environments (IDEs) are used to design, create, and deploy

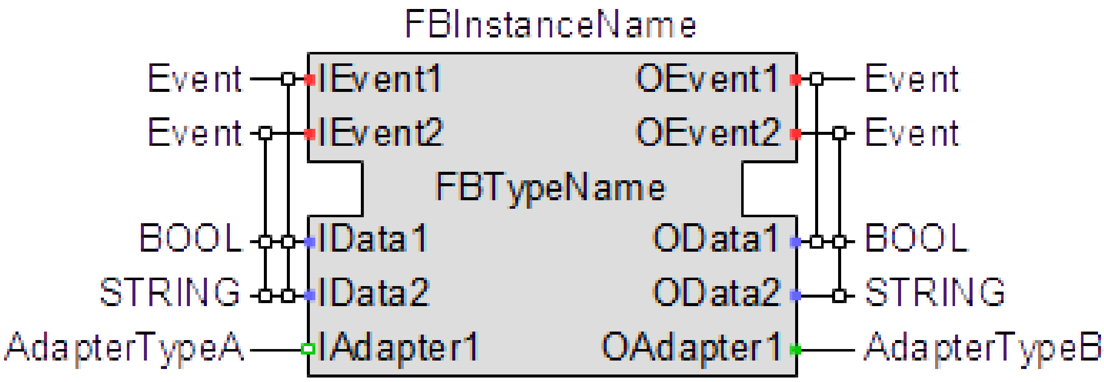

Figure 1.1: *IEC 61499* function block interface [5]

function block pipelines to CPPS. These IDEs offer an User Interface (UI) with features and tools that facilitate the development, testing, and deployment of *IEC 61499*-compliant control systems. They are graphical editors, like shown in Figure 1.2, for creating function block diagrams with simulation capabilities and additional functionalities to streamline the development process.

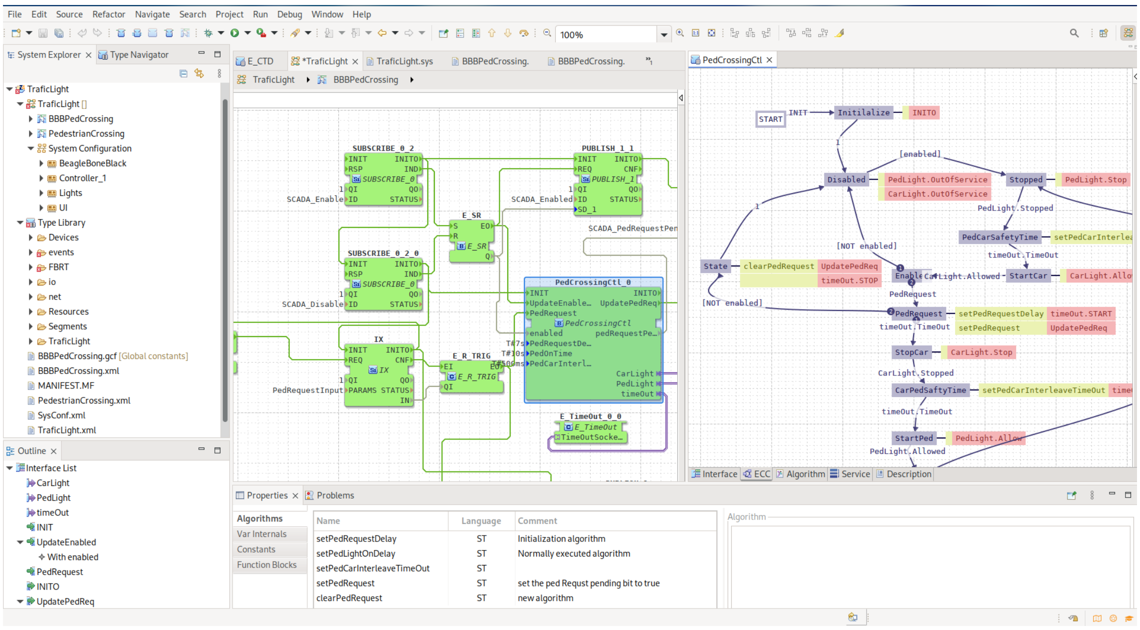

Figure 1.2: 4diac IDE [1]

1.2 Motivation

The evolution of CPPS has increased the demand for flexible and intelligent control solutions capable of integrating physical processes with digital technologies. FBD and the *IEC 61499* standard offer a structured, modular approach to building such distributed automation systems. To fully leverage these benefits, development tools must evolve alongside industrial needs, supporting more accessible, collaborative, and scalable workflows.

1.3 Problem

Existing *IEC 61499* development environments lack modern capabilities such as remote access, real-time collaboration, and seamless integration with cloud infrastructures. These limitations hinder efficient development, maintenance, and deployment of control applications within distributed CPPS. As a result, current tools fall short in supporting the flexibility and scalability required by certain modern industrial automation systems. Besides this, the complexity of these current IDEs rules them out for many simpler use cases where the time wasted on setting up and learning them is not justifiable.

1.4 Research Questions

This research seeks to investigate how modern web and distribution technologies can be leveraged to enhance the development, deployment, and operation of *IEC 61499*-compliant systems. This study is guided by the following main research question:

> How can we design IDEs that improve the development, deployment, and collaboration processes for *IEC 61499* applications in distributed CPPS?

To further refine the scope, the following sub-question is proposed:

> What architectural and technological requirements must be considered when designing an IDE for *IEC 61499*?

Chapter 2

Literature Review

This chapter reviews prior work related to *IEC 61499*-based development and the used IDEs. The goal is to better understand the landscape, which will then allow the critical review of the state of the art.

2.1 Methodology

A targeted literature review was carried out, focusing on studies addressing *IEC 61499* development tools, function blocks, CPPS, and development environments, to then critically reflect upon their findings to help the writing of this dissertation and the planning of the final product.

2.1.1 Search and Selection

The search was conducted in digital libraries *ScienceDirect*, *IEEE Xplore*, *ACM Digital Library*, among others, as well as research paper aggregators like *Semantic Scholar*, *Scopus* and *Web of Science* using combinations of the following terms:

- *IEC 61499*
- FBD
- IDE
- CPPS

Following the search, a manual review process was conducted to refine and validate the results. Titles and abstracts were first screened to understand the relevance of the articles with respect to this dissertation. The publications were then examined in greater detail through full-text review, the reference lists of selected papers were manually inspected to identify relevant works that may not have been captured by the initial keyword search.

Furthermore, terms related to the state of the art that were discovered mid search were also included in some queries. The objectives of doing this were to get a better personal understanding

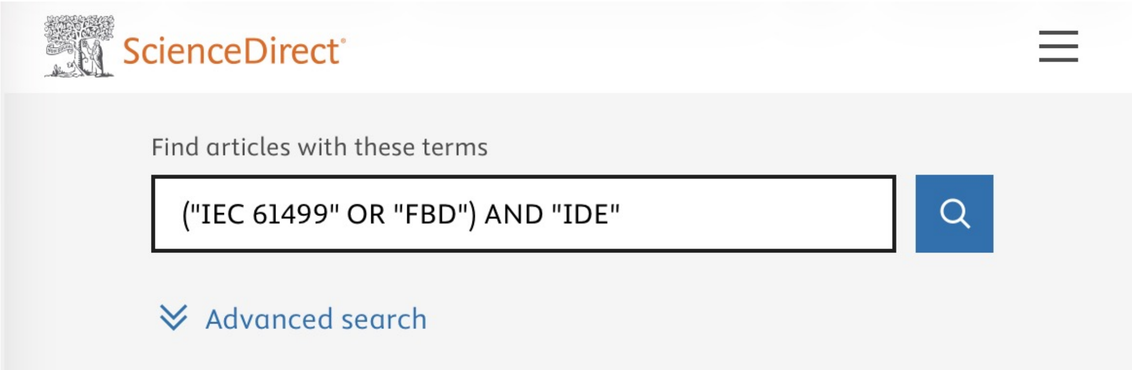

Figure 2.1: Example query on *ScienceDirect* with 136 results [11]

of the scene, as well as to find interesting references that could be included in this dissertation to backup the written claims. Some of the terms used were:

- 4diac
- collaboration

2.2 Related Work

It should be interesting to mention some papers that were used to get the knowledge used for writing this dissertation and which will be referenced throughout the whole document.

Monostori [7] writes in-depth about CPPS, while Torres, Gonçalves, and Pinto [13] discuss how *IEC 61499* can be leveraged for CPPS development. Zoitl et al. [17] demonstrated the use of the open-source *4DIAC* environment to develop modular and reusable *IEC 61499* applications, highlighting the benefits of flexibility and adaptability in industrial settings. Similarly, Pinto et al. [9] presented an industrial case study showing that function blocks oriented approaches improve system modularity and reduce development effort compared to traditional procedural programming. Some more recent studies explore web-based execution of FBD, Rohat and Popescu [10] as well as Lyu et al. [6] propose the creation of a web platform for *IEC 61499* development.

Chapter 3

State of the Art

This chapter provides a comprehensive analysis of the current landscape of development environments for CPPS and the *IEC 61499* standard. It categorises and critically examines the existing development tools, while looking for possible gaps in the ecosystem.

3.1 Ecosystem Summary

Currently, the development ecosystem is composed only by desktop-based IDEs. These tools are typically monolithic applications installed locally on engineering workstations.

3.1.1 Open-Source

The most prominent open-source implementation is the *Eclipse 4diac* IDE (based on the *Eclipse* framework) which supports a variety of RTEs such as *4diac FORTE* and *DINASORE*. This IDE is very feature rich at the cost of significant system resources and increased complexity, resulting in a steep learning curve, which makes it more difficult to use [15].

An alternative is *Jetbrains FBME* which is a much simpler modelling environment [12]. While easier to use than 4diac IDE, it still is a local-first development environment. The importance of this will be explained in Section 3.2.

Figure 3.1: *4diac* logo

3.1.2 Commercial

The commercial side is occupied by *EcoStruxure Automation Expert*, *ISaGRAF Workbench* and other development environments that usually have their own built-in proprietary RTE. This makes them often very complete production ready tools, but they are not very flexible and may not be the best fit for many needs.

3.2 Local-first development environments

3.3 Evolution of Web-Based Engineering Tools

The software engineering domain has seen a massive shift from desktop IDEs to cloud-native environments. This trend is now influencing the domain of Industrial Control Systems (ICS), and while recent studies have proposed a Service-Oriented Architectures (SOA) to integrate *IEC 61499* with cloud layers [13], the engineering tools themselves remain largely desktop-bound.

3.3.1 Web-Based IDEs in Software Development

In general software development, tools like *Visual Studio Code* have started offering web-based counterparts (as seen in Figure 3.2) to their desktop applications [14] in search of a more flexible user experience. In fact, newer development environments like *Zed* make it a key goal of their products to have web platform support [16].

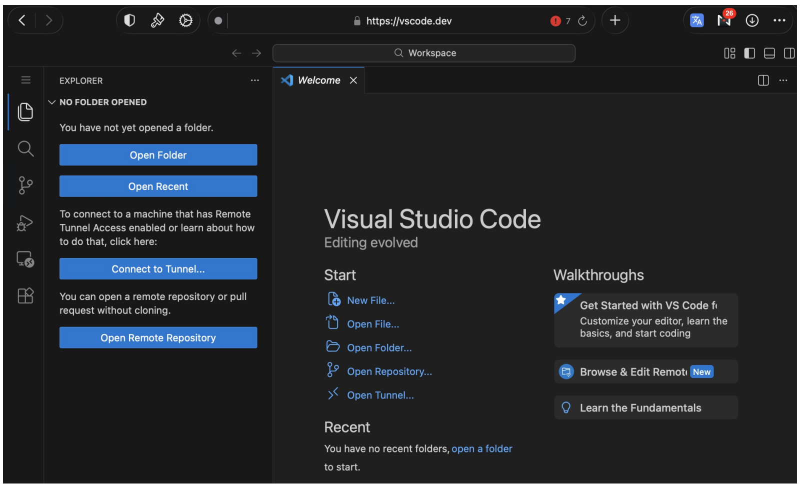

Figure 3.2: *VSCode Web* interface

3.4 Gap Analysis

Current *IEC 61499* tools face significant limitations that hinder modern CPPS development:

- **Platform Dependency:** Existing solutions are monolithic desktop applications, lacking the accessibility and portability of web-based environments.
- **Resource Intensity:** Current options are heavyweight and complex, creating a high barrier to entry compared modern development tools of other purposes.
- **Web Trends:** Despite industry trends, there is currently no mature, lightweight web interface for FBD, leaving ICS engineering behind the curve of cloud-native software development.

Chapter 4

Proposed Solution

To address the limitations identified in existing *IEC 61499* development environments, this dissertation proposes the design and implementation of a lightweight, web-based IDE for modelling *IEC 61499*-compliant applications. This work seeks to bridge the gap between modern software engineering practices and the current state of industrial automation tooling for CPPS. A web-based IDE solves many of the problems mentioned in Chapter 3, addressing the identified gap.

The main reason to propose using web technologies is to solve the issues that affect any local-first development environment. By having the IDE be hosted by a centralised server, the following is achieved:

- **Ease of setup**. Instead of each team member having to install a resource heavy program and set it up with the team's code and configurations, a single server is set up and the team members simply have to connect to it via their browser.
- **Ease of deployment**. Only the server needs to be aware of all of the target RTEs and have connection to them. When a team member wants to deploy, the request will be routed through the server, acting as the middleware, removing the need for each development machine to have connection with the target RTEs.
- **Real-time collaboration**. All data is kept and directly modified in the server itself, instead of requiring a periodic synchronisation system like *git*. FBD coded programs, since they are meant for visual editing, need to be stored in formats that don't easily handle merge conflicts. Real-time combats this by keeping track of atomic changes, instead of blocks of text.

4.1 Approach

The approach begins with the derivation of functional and non-functional requirements from the *IEC 61499* standard and an analysis of existing desktop-based tools. Core requirements include support for function block types, event and data connections, and deployment to compliant runtime environments, namely *DINASORE*. Non-functional requirements emphasise platform independence, reduced complexity, usability, and extensibility, addressing the resource intensity and accessibility issues of current solutions. The use of modern web technologies further allows the inclusion of collaborative features and scalable workflows, which are largely absent from current *IEC 61499* tools.

The effectiveness of the approach is assessed through the implementation of a functional prototype and its direct comparison with established desktop-based IDEs like *4diac*, focusing on usability, accessibility, and workflow efficiency. This evaluation demonstrates the feasibility of applying web-based principles to *IEC 61499* development and their potential benefits for distributed CPPS.

4.2 Methods

This section describes the research methodology adopted to address the research questions defined in this dissertation. Given the practical and design-oriented nature of the problem, the methodology focuses on the design, implementation, and evaluation of a piece of software. The chosen methods are appropriate for investigating how web-based technologies can enhance *IEC 61499* development environments, as they allow both the realisation of a concrete solution and the systematic assessment of its capabilities relative to existing tools.

4.2.1 Research Design

This research follows a qualitative, design-oriented research approach. The core of the research consists of the design and implementation of a functional prototype of the solution. Rather than aiming for statistical generalisation, the study focuses on demonstrating feasibility and identifying qualitative improvements over existing desktop-based environments. The prototype serves as a research artifact used to explore architectural decisions, usability considerations, and workflow implications.

4.2.2 Evaluation

Evaluation is conducted through qualitative analysis and comparative assessment against established *IEC 61499* IDEs. Criteria such as accessibility, usability, feature coverage, and support for collaborative workflows are used to assess how effectively the proposed solution addresses the limitations identified in the literature and state of the art.

4.3 Workplan

Figure 4.1 outlines the planned work for this dissertation. The schedule is organised into three main phases: Experimentation, Code Development, and Deliveries. During the Experimentation phase, there will be a focus on studying the standard and running experiments to establish a solid technical foundation. The Code Development phase overlaps partially with Experimentation, ensuring that the software evolves in parallel with the experimental insights. Finally, the Deliveries phase spans the entire project duration and includes writing the dissertation and preparing the final presentation, ensuring that all outputs are documented and communicated effectively.

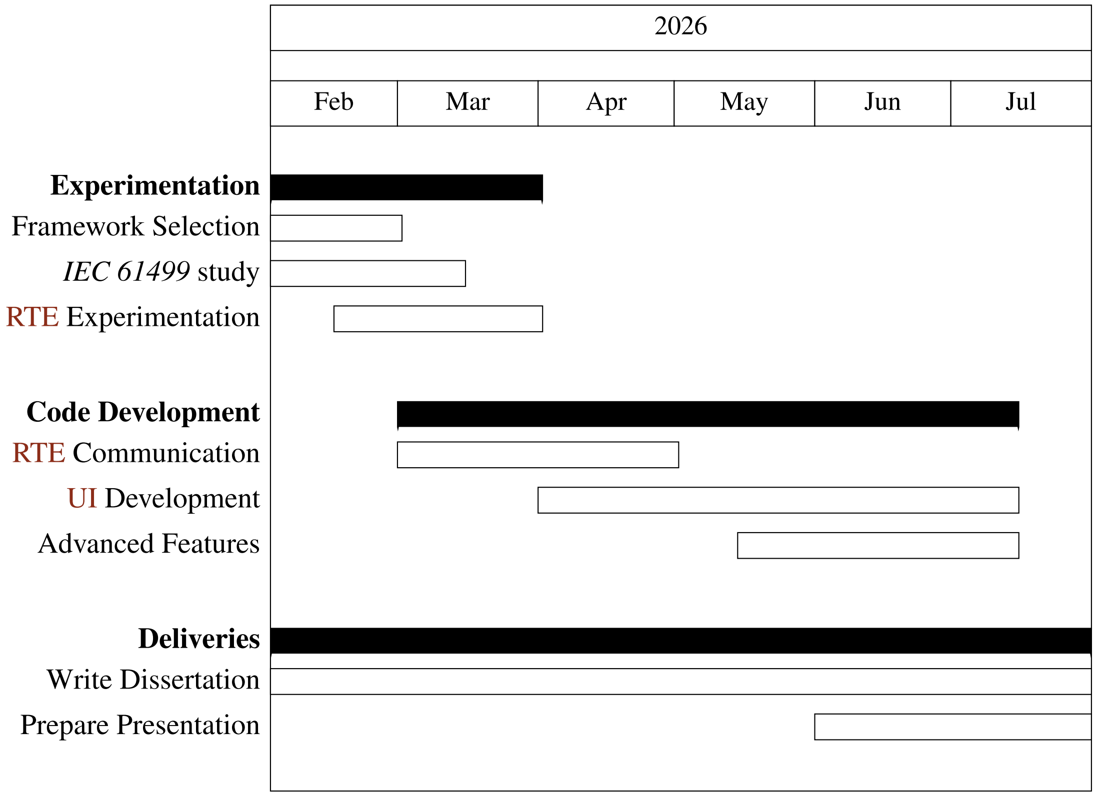

Figure 4.1: Gantt chart timeline for the proposed work

Chapter 5

Prototype Development

To be able to prove the proposed solution is viable and will successfully address the gap identified, a prototype needs to be developed and afterwards validated.

5.1 Runtime Environment

Out of all RTEs compatible with *IEC 61499*, this prototype will focus on supporting *DINASORE* (Dynamic INtelligent Architecture for Software and MOdular REconfiguration). It is much less complex than others, making it ideal for a proof of concept IDE. As an open-source project, *DINASORE* can be freely inspected and studied, which is particularly valuable in a research and prototyping context where flexibility is prioritised over real-time performance guarantees. This ease of study rules out the use of commercial RTEs like *EcoStruxure Automation Expert's "dPAC"* and *ISaGRAF Runtime*. Other open-source RTEs were considered, but none was as well-suited as *DINASORE*. *4diac FORTE* for example was way too extensive and complicated to be able to easily study its source code.

By default, *DINASORE* uses the port *61499* to receive commands from the IDE and send responses. This is important to note as it is the port that will be used as example throughout this document. The exact details of the communication implementation will be explained further in Section 5.5.

Figure 5.1: *DINASORE* logo

5.2 Server

The design begins by setting up a server that will serve the IDE interface as a webpage, store and manage all the data and handle communication with the RTEs. Taking into account the features we plan on implementing, *JavaScript* running on *Node.js* were chosen due to the extensive library ecosystem. For real-time collaboration we will need a library to run in both the server and the frontend, since *JavaScript* runs natively in the browser, it further reinforces that it should also be used in the server. The library choice will be explained in depth in Section 5.4, proving the previous train of thought.

Being the middleware, there are 4 different types of network communications that the server will need to handle, each with it's own requirements:

- **Serve webpage**: Communicates via *GET* requests to deliver the *html*, *css* and *js* files that make up the IDE interface.
- **Real-Time Collaboration**: Constant atomic updates via a library.
- **Requested deployment**: A team member sends a *POST* request, that tells the server to deploy.
- **Deployment sockets**: Communicates via sockets with each RTE to send commands and receive the corresponding responses.

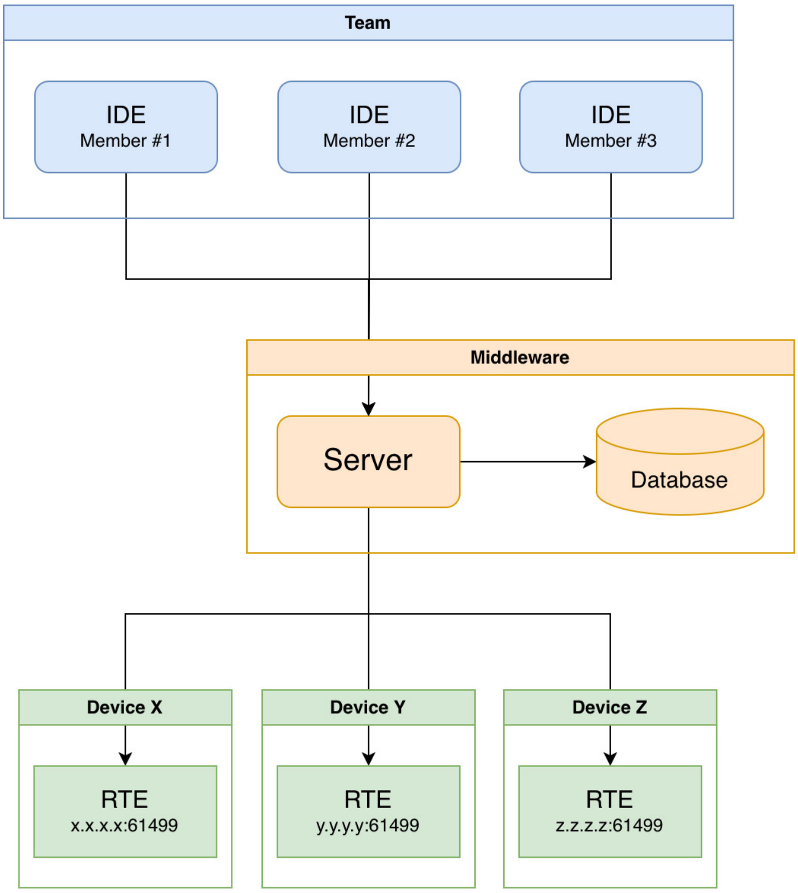

Figure 5.2: Communication through the server

5.3 Frontend

The frontend is the UI that will be used to create and modify the FBD programs.

Each subsection will first explain and describe what an IDE should implement and follow by showing how the prototype solved the proposed requirements.

5.3.1 Devices Configuration View

In the interest of distributed CPPS, *IEC 61499* allows function blocks to be mapped to different *DINASOREs*, so we need a clean way to define the resources that are available. Both the *DINASORE* port and the *SSH* port need to be defined for each defined since deployment communication will be made through both.

Each device should have several fields that the team can configure:

- **Name**: Human friendly name to quickly identify what device this is
- **Address**: The address of the device
- **DINASORE port**: The port where *DINASORE* is running

- **Color**: The color associated with this *DINASORE* instance, providing a visual distinction in the *Graph View* between function blocks mapped to different instances.
- **SSH port**: The open *SSH* port of the device.
- **SSH user** and **SSH password**: The *SSH* login credentials.
- **SSH path to DINASORE**: Path in the *SSH* filesystem pointing to where *DINASORE* is located.

With the existence of the middleware, this solution removes the requirement for each team member to configure the available *DINASORE* instances in their own IDE, and instead keeps a shared configuration in the server database.

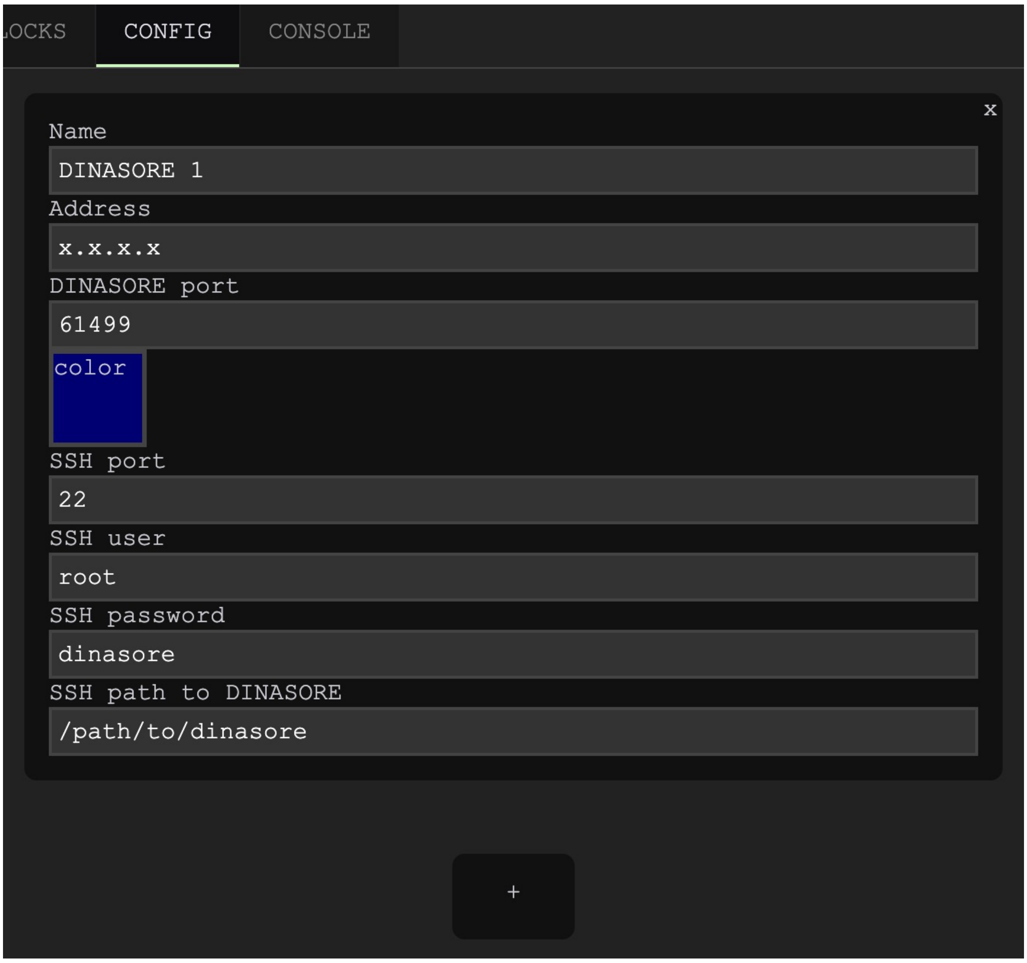

Figure 5.3: Devices Configuration View

5.3.2 Function Blocks Editor View

This is where individual function blocks are defined. It allows the team to specify the interface of each function block, its input and output data ports and events, as well as program the internal behaviour. This separation of interface and implementation follows the *IEC 61499* model of encapsulation, where a function block exposes a well-defined interface while keeping its internal logic self-contained.

Like in the other RTEs, in *DINASORE* the interface of the function block is defined in a *xml* format file with a *.fbt* extension, as seen in Listing 5.1 which results in Figure 5.4.

```xml
<FBType Name="NEW_FB_1">
  <InterfaceList>
    <EventInputs>
      <Event Name="INIT" Type="Event"/>
      <Event Name="READ" Type="Event">
        <With Var="VARIABLE_1"/>
        <With var="VARIABLE_2"/>
        <With var="VARIABLE_3"/>
      </Event>
    </EventInputs>
    <EventOutputs>
      <Event Name="INIT_O" Type="Event"/>
      <Event Name="READ_O" Type="Event">
        <With Var="DATA_1"/>
      </Event>
    </EventOutputs>
    <InputVars>
      <VarDeclaration Name="VARIABLE_1" Type="STRING"/>
      <VarDeclaration Name="VARIABLE_2" Type="REAL"/>
      <VarDeclaration Name="VARIABLE_3" Type="INT"/>
    </InputVars>
    <OutputVars>
      <VarDeclaration Name="DATA_1" Type="STRING"/>
    </OutputVars>
  </InterfaceList>
</FBType>
```

Listing 5.1: Example *xml* definition of a function block

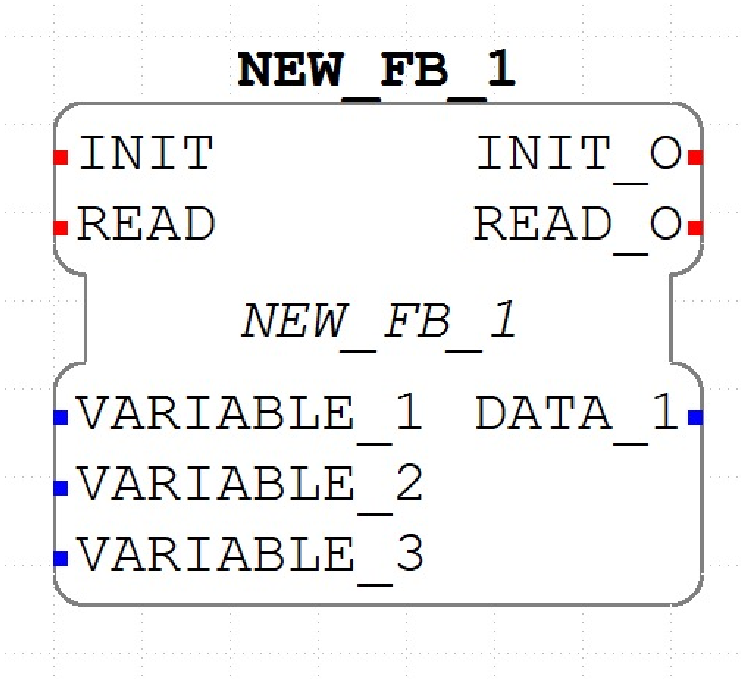

Figure 5.4: Example function block defined via *xml*

The internal behaviour of the function blocks in *DINASORE* is programmed in *Python*.

```
class NEW_FB_1:
  def __init__(self):
    pass

  def schedule(self, event_name, event_value, variable_1, variable_2, variable_3):
    if event_name == "INIT":
      return [event_value, None, ""]

    elif event_name == "READ":
      return [None, event_value, ""]
```

Listing 5.2: Example *Python* code of a function block

For the IDEs to be able to detect and use the function block, both the *.fbt* file and the *.py* file need to be placed in a specific folder, right next to each other, like shown in Figure 5.5. In this prototype this is handled automatically in the server database, removing the need to manually move files like it is needed in *4diac IDE*.

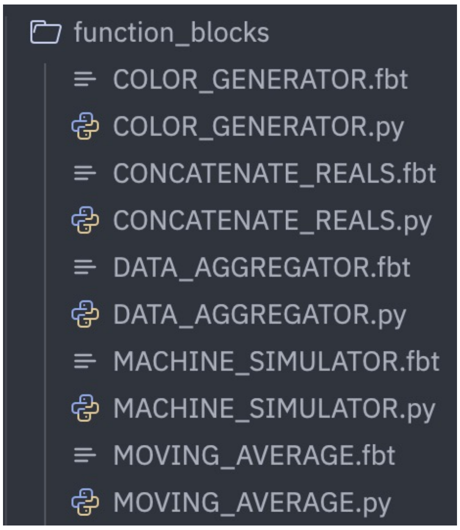

Figure 5.5: Example function block defined via *xml*

This view contains a split view text editor: a side for the *.fbt* file and the other for the *.py* file. It would be possible to create a visual editor for designing the interface, instead of having to write *xml* by hand, but the added complexity would not be justified at this prototyping stage. The *xml* format is sufficiently structured that a text editor provides an adequate experience, and a dedicated visual interface remains a natural direction for future development.

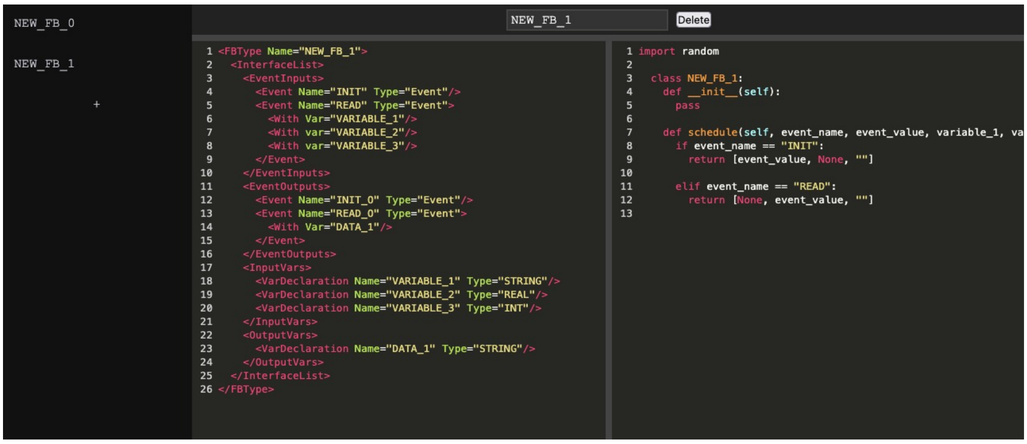

Figure 5.6: Function Blocks Editor View

5.3.3 Graph View

This view is the primary workspace for composing the application. Function blocks are placed on the canvas and connected together by drawing connections between their data and event ports, forming the application network. Event ports are commonly color coded red, while data ports are color coded blue.

In *DINASORE*, all *INIT* event ports are by default assigned to *EMB_RES*, the default resource that sends an event when the application is started. This removes the need to connect the *INIT* event ports, but the *READ* event port is not assigned by default and therefore needs to be connected in every function block. The `schedule()` function will only be called and generate output when an event is sent to the *READ* event port. The described event driven execution model is what allows an explicit specification of the execution order of function blocks. Periodic out of event execution of function blocks is also possible with the implementation of a looping function block that triggers events [5].

A function block will have some input and output data ports, each with its own associated data type. For example, an *INT* input port will not accept data from a *WORD* output port, and therefore should not be able to connect to it. It is also possible to set a default value for an input if no output is connected to it, there is an editable string field for this which gets converted to the expected data type by the RTE. The data types supported by *IEC 61499* are derived from and defined by *IEC 61131-3*.

Type	Description	Size
`BOOL`	Boolean	1-bit
`SINT`	Short integer	8-bit
`INT`	Integer	16-bit
`DINT`	Double integer	32-bit
`LINT`	Long integer	64-bit
`REAL`	Floating point	32-bit
`LREAL`	Long floating point	64-bit
`STRING`	ASCII string	Variable
`WSTRING`	Unicode string	Variable
`TIME`	Time interval	
`DATE`	Date	

Table 5.1: IEC 61131-3 Data Types [4]

Each function block can also be mapped to a *DINASORE* instance, with the color coding defined in the Devices Configuration View providing an immediate visual indication of how the application is distributed across the available devices.

This system was achieved in the prototype with the use of the *LiteGraph.js* library, a graph node editor for the browser. A lot of other libraries were considered, but this one was chosen due to the requirements of the proposed solution. Let us discuss the alternatives that were considered to understand why this choice is important:

- **Rete.js**: A very flexible and complete editor framework, but the added complexity makes it harder to integrate the desired features. It is also a more general purpose editor, meaning the multi-input and multi-output function blocks with specific data types are not as well supported.

- **xyflow**: A highly customizable framework composed of React Flow and Svelte Flow. While it provides excellent visual customisation and integration with frontend frameworks, it suffers from the same problems as *Rete.js*.
- **Drawflow**: A lightweight and simplistic library with a focus on data flow execution. Although simple to integrate, it lacks many features required for designing *IEC 61499* function blocks, such as robust type handling and multi-input and multi-output.
- **Baklava.js**: With a clean architecture and flexibility, this would be a good alternative, but since it specifically targets *Vue* and requires a more intricate setup, it was not the most appropriate choice for a prototype.
- **GoJS**: Very powerful commercial alternative. It offers advanced functionality and flexibility, but its licensing model make it less suitable for lightweight research prototypes and open development environments.
- **Node-RED**: A flow based programming environment. While conceptually similar, its architecture is more focused on runtime execution than on being a visual editor of structured data.

Among these alternatives, *LiteGraph.js* presented the best balance between simplicity, flexibility and ease of integration. Its builtin support for typed inputs and outputs, lightweight architecture and direct focus on visual data flow programming made it especially suitable for the requirements of the proposed solution. Even so, it was necessary to implement custom behaviour on top of some functions of *LiteGraph.js* to achieve the desired behaviour. The choice of the graph library is very important, since the requirements of *IEC 61499* differ significantly from those of most conventional graph editors, as such, special care must be taken to ensure that these requirements are properly supported.

5.3.4 Console View

This view will display the console logs that are generated when deploying. The contents of it will be discussed in detail in Section 5.5.

5.4 Real-Time Collaboration

One of the main goals of the proposed solution is to support collaborative development of *IEC 61499* applications. Since CPPS projects are often developed by multiple team members simultaneously, the code collaboration system must accommodate for this. Traditional version control systems such as *Git* are not well suited for collaborative editing of structured graph based data represented in formats such as *json* or *xml*. Although these formats are text based, small logical modifications in the application model can produce large text changes due to formatting changes, element reordering or nested structure updates. As a result, merge conflicts become frequent and difficult to resolve, especially when multiple team members edit the a graph at the same time. This limitation makes real time collaborative editing impractical, motivating the use of synchronisation approaches based on shared data structures and conflict free replication mechanisms such as Conflict-free Replicated Data Types (CRDTs).

In this prototype the collaborative editing system was implemented using *Yjs*, a CRDT framework designed for distributed collaborative applications. CRDT based systems allow users to concurrently edit shared data structures while automatically resolving conflicts in a deterministic manner.

The graph structure in *LiteGraph.js* representing the FBD application is synchronised through shared *Yjs* data structures. Operations such as creating function blocks, deleting connections, modifying inputs or moving nodes are propagated in real time to all connected clients. Each user therefore observes the same graph state with minimal delay. To integrate collaboration into this editor, the internal state changes of the graph were connected to the shared *Yjs* document. Whenever a local modification occurs in the graph, the corresponding operation is serialized and applied to the shared collaborative state. Likewise, remote updates received from other users are reflected back into the local graph editor instance.

The serialized data for the graph structure can be divided into 2 maps (using the `Y.Map` structure), one for the nodes (shown in Listing 5.3) and one for the links (shown in Listing 5.4). In the nodes map, since the `title` of each node can be modified and the `type` can be the same across multiple nodes, we need to define a unique persistent id for each. Besides these, we need to also store the position of the node defined as `x` and `y` coordinates, the id of the *DINASORE* that this node is `mappedto` and the default `inputs` in case some data input slot is not being fed data from any link. As for the links map, each link must also have a unique persistent id, and then store the origin (represented by the `origin_id` node and the `origin_slot`) and the target (represented by the `target_id` node and the `target_slot`)

```
{
  'node_id_1': {
    type: 'SENSOR_SIMULATOR',
    title: 'SENSOR_SIMULATOR_1',
    x: 139, y: 526,
    mappedto: 'dinasore_id_1',
    inputs: { OFFSET: '' }
  },
  'node_id_2': {
    type: 'MOVING_AVERAGE',
    title: 'MOVING_AVERAGE_1',
    x: 487, y: 551,
    mappedto: 'dinasore_id_1',
    inputs: { WINDOW: '5', VALUE: '' }
  }
}
```

Listing 5.3: Structure of the Nodes serialized map

```
{
  'link_id_1': {
    origin_id: 'node_id_1',
    origin_slot: 'VALUE',
    target_id: 'node_id_2',
    target_slot: 'VALUE'
  },
  'link_id_2': {
    origin_id: 'node_id_1',
    origin_slot: 'READ_0',
    target_id: 'node_id_2',
    target_slot: 'RUN'
  }
}
```

Listing 5.4: Structure of the Links serialized map

5.5 Deployment

The deployment process is responsible for taking the application defined in the *Graph View* and bringing it to life across the configured *DINASORE* instances. It is broken down into a sequence of steps, each of which is described in the following subsections.

5.5.1 Synchronise with DINASOREs

The first step is to ensure that each *DINASORE* instance has access to the function blocks that it will need to run the application. This is done by sending all the *.fbt* and corresponding *.py* files in the database, to every device. To achieve this we can use the `synchronize.py` script that accompanies the *DINASORE* source code. A configuration file needs to be provided that tells the script where to send the function blocks to, like shown in Listing 5.5. First, `master-fbs-path` dictates where in the server filesystem the function block files that will be sent are located, then an array of `dinasores` with the configurations defined in the *Devices Configuration View*. The `port` used here will be the *SSH* port, the *DINASORE* must have an open *SSH* channel that the server is able to connect to via this port.

```
{
  master-fbs-path: 'sync/fbs',
  dinasores: [
    {
      address: 'x.x.x.x',
      port: 22,
      username: 'root',
      password: 'dinasore',
      dinasore-path: '/root/dinasore'
    },
    {
```

```
      address: 'y.y.y.y',
      port: 22,
      username: 'root',
      password: 'dinasore',
      dinasore-path: '/root/dinasore'
    }
  ]
}
```

Listing 5.5: Sync configuration

This file is generated from the configured fields in the *Devices Configuration View*. When the script is ran, it will read the contents of this file in order to connect and send the files to each *DINASORE*.

5.5.2 Send Commands

Once the function block files are in place, the server will send the necessary commands to each *DINASORE* instance to build and start the application. This includes creating the function block instances, writing the initial input values, establishing the connections between event and data ports and finally sending the start command to begin execution. The order in which these messages are issued is important, as connections can only be established between existing nodes. The following two subsections describe how the messages are constructed and what they contain.

5.5.2.1 Build messages

Before the messages can be sent, they need to be built into the format that *DINASORE* is expecting. It consists of a two part message: the first part refers to the configuration name, explained later in Section 5.5.2.2, while the second is the actual message payload containing the wanted command. Each of these parts are preceded by a 3-byte header composed of a `0x50` byte followed by 2 bytes indicating the size of the corresponding content. A representation of this format is presented in Figure 5.7.

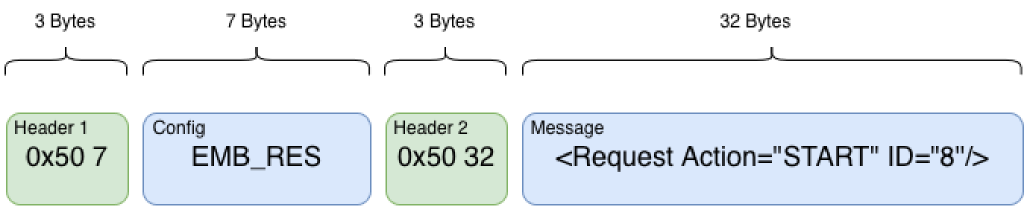

Figure 5.7: *DINASORE* communication format

The *4diac_simulator.py* file, part of the *tests/* folder of *DINASORE*, implements this method in *Python* as shown in Listing 5.6. For this prototype, it was necessary to implement it in *JavaScript*, as shown in Listing 5.7.

```python
def build_message(message_payload, configuration_name):
  # build the first part of the header
  hex_input = '{:04x}'.format(len(configuration_name))
  second_byte = int(hex_input[0:2], 16)
  third_byte = int(hex_input[2:4], 16)
  response_len = struct.pack('BB', second_byte, third_byte)
  response_header_1 = b''.join([b'\x50', response_len])

  # build the second part of the header
  hex_input = '{:04x}'.format(len(message_payload))
  second_byte = int(hex_input[0:2], 16)
  third_byte = int(hex_input[2:4], 16)
  response_len = struct.pack('BB', second_byte, third_byte)
  response_header_2 = b''.join([b'\x50', response_len])

  # join the 3 parts
  response = b''.join([response_header_1, configuration_name, response_header_2,
      message_payload])
  return response
```

Listing 5.6: build_message() in Python

```javascript
function build_message(message_payload, configuration_name) {
  const configBuffer = Buffer.from(configuration_name, 'utf-8');
  const messageBuffer = Buffer.from(message_payload, 'utf-8');

  // Header 1: [0x50][len_hi][len_lo]
  const header1 = Buffer.alloc(3);
  header1[0] = 0x50;
  header1.writeUInt16BE(configBuffer.length, 1);

  // Header 2: [0x50][len_hi][len_lo]
  const header2 = Buffer.alloc(3);
  header2[0] = 0x50;
  header2.writeUInt16BE(messageBuffer.length, 1);

  return Buffer.concat([
    header1,
    configBuffer,
    header2,
    messageBuffer
  ]);
}
```

Listing 5.7: build_message() in JavaScript

5.5.2.2 Send Messages

Now that we have defined how to build the messages, we can explore the sequence of commands that need to be sent to each *DINASORE* instance and what configuration name should be used in each. The server connects to each instance and transmits these messages sequentially. The order of message transmission is critical, as connections can only be established between existing nodes. The configuration name used in each message will be determined by whether the command is bound to any resource. In *DINASORE* the existing resource is *EMB_RES* as mentioned before.

First we need to send the 2 commands in Listing 5.8 to every device. A `QUERY` request and a `CREATE` request that will create a *EMB_RES* resource which will be automatically connected to the *INIT* port of all other function block nodes to initialize them. Both these commands will have an empty configuration name, as *EMB_RES* has not been created yet and therefore cannot be bounded to.

```
<Request Action="QUERY" ID="0">
  <FB Name="*" Type="*"/>
</Request>

<Request Action="CREATE" ID="1">
  <FB Name="EMB_RES" Type="EMB_RES"/>
</Request>
```

Listing 5.8: Setup deployment commands

Next, depending on which commands are mapped to each machine, we `CREATE` all the mapped nodes and `WRITE` the initial input values with the commands in Listing 5.9. The configuration name in these and all following commands will be "*EMB_RES*" as they need to be bound to it in order to be initialised and ran.

```
<Request Action="CREATE" ID="2">
  <FB Name="SENSOR_SIMULATOR" Type="SENSOR_SIMULATOR"/>
</Request>

<Request Action="WRITE" ID="3">
  <Connection Destination="SENSOR_SIMULATOR.OFFSET" Source="12"/>
</Request>

<Request Action="CREATE" ID="4">
  <FB Name="MOVING_AVERAGE" Type="MOVING_AVERAGE"/>
</Request>

<Request Action="WRITE" ID="5">
  <Connection Destination="MOVING_AVERAGE.WINDOW" Source="5"/>
</Request>
```

Listing 5.9: `CREATE` nodes deployment commands

Then the connections are created between the mapped nodes. A `Source` a `Destination` must be provided, with the format `"NODE_NAME.SLOT_NAME"`, using the commands in Listing 5.10.

```xml
<Request Action="CREATE" ID="6">
  <Connection Destination="SENSOR_SIMULATOR.READ_0" Source="MOVING_AVERAGE.RUN"/>
</Request>

<Request Action="CREATE" ID="7">
  <Connection Destination="SENSOR_SIMULATOR.VALUE" Source="MOVING_AVERAGE.VALUE"/>
</Request>
```

Listing 5.10: `CREATE` links deployment commands

Finally the `START` command in Listing 5.11 can be issued, which will request for the start of the program execution. For this command the configuration name used will also be "*EMB_RES*" as it is this resource that will begin the execution of the program.

```xml
<Request Action="START" ID="8"/>
```

Listing 5.11: `START` deployment command

After each request is sent, a response will be returned by the *DINASORE*. It is important for the IDE to listen for these responses and only send the next request once the response corresponding to the previous request is received. The responses have the format shown in Listing 5.12. The `ID` of the response will be the same of the associated request. The format of the responses is the same as the requests and the configuration name that is used is empty.

```xml
<Response ID="8"/>
```

Listing 5.12: `Response` after a deployment command is sent

5.5.3 Console

To give the team visibility into the deployment process right from the IDE, without having to log into the server, a console is provided in the *Console View* that displays a live log of each action taken during deployment. There are 2 main things being logged:

- **Synchronise**: The output of running the *Python* script is displayed so that the team see that the *SSH* communication is being done properly.
- **Send Commands**: As commands are sent and responses are received, they are displayed, allowing the team to make sure each command is formulated correctly, being processed and a response is issued.

5.5.4 Watch

Once the application is running, the values of the node outputs are continuously retrieved from the *DINASORE* instances and displayed in the *Graph View*, providing immediate feedback on the state of the running application. This is particularly useful for debugging and validating the behaviour of the application during development. In order to get these values, the IDE must tell *DINASORE* instances which output slots it wants to watch and then request the values. In this prototype, for simplicity, all data outputs are watched and we need to send a watch creation request for each. Note in Listing 5.13 that the `ID` is reset after deploying.

```
<Request ID="0" Action="CREATE">
  <Watch Source="SENSOR_SIMULATOR.VALUE" Destination="*" />
</Request>

<Request ID="1" Action="CREATE">
  <Watch Source="MOVING_AVERAGE.VALUE_MA" Destination="*" />
</Request>
```

Listing 5.13: `CREATE` watches after deploying

After creating the watches we can send a request to receive the values using the command in Listing 5.14. This will return the response in Listing 5.15, which by parsing the values, allows the prototype to display them in the *Graph View*, on the right side of the corresponding output slots, demonstrated in Figure 5.8.

```
<Request ID="2" Action="READ">
  <Watches/>
</Request>
```

Listing 5.14: Request to `READ` watches

```
<Response ID="2">
  <Watches>
    <Resource name="EMB_RES">
      <FB name="SENSOR_SIMULATOR">
        <Port name="VALUE">
          <Data value="13.75" />
        </Port>
      </FB>
      <FB name="MOVING_AVERAGE">
        <Port name="VALUE">
          <Data value="13.3" />
        </Port>
      </FB>
    </Resource>
  </Watches>
```

```
</Response>
```

Listing 5.15: Response to `READ` watches

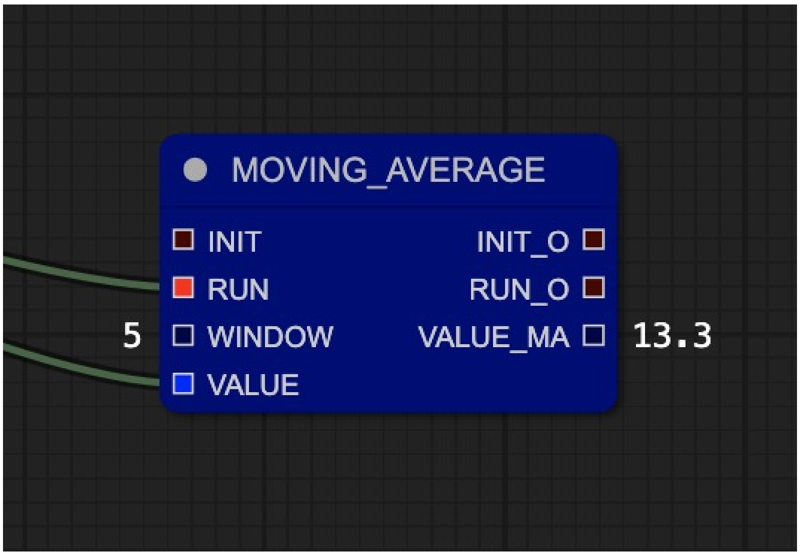

Figure 5.8: Watching an output slot in the *Graph View*

5.6 Distribution

Chapter 6

Results and Analysis

Chapter 7

Conclusions

References

[1] *4diac IDE Website*. URL: https://eclipse.dev/4diac/4diac_ide/.

[2] Young-Jo Cho et al. “Function Block Diagram Approach for Manufacturing Process Control Programming”. en. In: *IFAC Proceedings Volumes* 26.2 (July 1993), pp. 461–464. ISSN: 14746670. DOI: 10.1016/S1474-6670(17)48510-7. URL: https://linkinghub.elsevier.com/retrieve/pii/S1474667017485107 (visited on 01/28/2026).

[3] Marcelo V. Garcia et al. “Engineering tool to develop CPPS based on IEC-61499 and OPC UA for oil&gas process”. en. In: *2017 IEEE 13th International Workshop on Factory Communication Systems (WFCS)*. Trondheim, Norway: IEEE, May 2017, pp. 1–9. ISBN: 978-1-5090-5788-7. DOI: 10.1109/WFCS.2017.7991969. URL: http://ieeexplore.ieee.org/document/7991969/ (visited on 01/30/2026).

[4] *IEC 61131-3*. en. Page Version ID: 1339669093. Feb. 2026. URL: https://en.wikipedia.org/w/index.php?title=IEC_61131-3&oldid=1339669093 (visited on 05/11/2026).

[5] *IEC 61499 Wikipedia*. URL: https://en.wikipedia.org/wiki/IEC_61499.

[6] Tuojian Lyu et al. “Towards Web Platform for Cloud-Based Virtual Commissioning of IEC 61499 Distributed Automation Systems”. en. In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. Padova, Italy: IEEE, Sept. 2024, pp. 1–4. ISBN: 979-8-3503-6123-0. DOI: 10.1109/ETFA61755.2024.10710351. URL: https://ieeexplore.ieee.org/document/10710351/ (visited on 12/17/2025).

[7] László Monostori. “Cyber-physical Production Systems: Roots, Expectations and R&D Challenges”. en. In: *Procedia CIRP* 17 (2014), pp. 9–13. ISSN: 22128271. DOI: 10.1016/j.procir.2014.03.115. URL: https://linkinghub.elsevier.com/retrieve/pii/S2212827114003497 (visited on 01/28/2026).

[8] Linna Pang et al. “Formal verification of function blocks applied to IEC 61131-3”. en. In: *Science of Computer Programming* 113 (Dec. 2015), pp. 149–190. ISSN: 01676423. DOI: 10.1016/j.scico.2015.10.005. URL: https://linkinghub.elsevier.com/retrieve/pii/S0167642315002981 (visited on 01/29/2026).

[9] Gonçalo Pinto et al. “Digital Factory: An Industrial Case Study for Function Block-Oriented Development”. en. In: *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. Aveiro, Portugal: IEEE, Oct. 2023, pp. 01–06. ISBN: 979-8-3503-1161-7. DOI: 10.1109/WF-IoT58464.2023.10539486. URL: https://ieeexplore.ieee.org/document/10539486/ (visited on 01/29/2026).

[10] Oana Rohat and Dan Popescu. “Remote web-based execution of IEC 61499 function blocks”. en. In: *Proceedings of the 2014 6th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*. Bucharest, Romania: IEEE, Oct. 2014, pp. 33–38. ISBN: 978-1-4799-5478-0 978-1-4799-5479-7. DOI: 10.1109/ECAI.2014.7090220. URL: http://ieeexplore.ieee.org/document/7090220/ (visited on 01/28/2026).

[11] *ScienceDirect*. URL: https://www.sciencedirect.com.

[12] Radimir Sorokin, Sandeep Patil, and Valeriy Vyatkin. “Novel development tool for IEC 61499 based on domain-specific languages”. en. In: *IFAC-PapersOnLine* 55.2 (2022), pp. 439–444. ISSN: 24058963. DOI: 10.1016/j.ifacol.2022.04.233. URL: https://linkinghub.elsevier.com/retrieve/pii/S2405896322002348 (visited on 01/28/2026).

[13] Tomás Torres, Gil Gonçalves, and Rui Pinto. “Literature Review on Cloud-Based Service-Oriented Architecture for IEC 61499 Distributed Control Systems:” en. In: *Proceedings of the 15th International Conference on Cloud Computing and Services Science*. Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 174–181. ISBN: 978-989-758-747-4. DOI: 10.5220/0013293400003950. URL: https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0013293400003950 (visited on 01/28/2026).

[14] *VSCode Web*. URL: https://code.visualstudio.com/docs/setup/vscode-web.

[15] Thomas Weber, Alois Zoitl, and Heinrich Hußmann. “Usability of Development Tools: A CASE-Study”. en. In: *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. Munich, Germany: IEEE, Sept. 2019, pp. 228–235. ISBN: 978-1-7281-5125-0. DOI: 10.1109/MODELS-C.2019.00037. URL: https://ieeexplore.ieee.org/document/8904489/ (visited on 01/30/2026).

[16] *Zed Roadmap*. URL: https://zed.dev/roadmap.

[17] Alois Zoitl, Thomas Strasser, and Gerhard Ebenhofer. “Developing modular reusable IEC 61499 control applications with 4DIAC”. en. In: *2013 11th IEEE International Conference on Industrial Informatics (INDIN)*. Bochum, Germany: IEEE, July 2013, pp. 358–363. ISBN: 978-1-4799-0752-6. DOI: 10.1109/INDIN.2013.6622910. URL: http://ieeexplore.ieee.org/document/6622910/ (visited on 01/29/2026).